VecBrierLM Experiment Report

Evaluating Chemical Question Embedding Quality via CALM-Inspired Brier Score

Dataset	dataset_without_preference.jsonl	N = 1787 questions
Embedding Model	all-MiniLM-L6-v2 (384 dim)
Experiment Scale	8 Token Methods x 8 Chunk Methods = 64 combinations
Evaluation	VecBrierLM (Brier@k=1, 10, 50, full) — no API, local only
Reference	CALM: Continuous Autoregressive LM (Shao et al., arXiv:2510.27688)
Date	2026-03-28

1. Method Background: CALM Paper

1.1 What is CALM?

CALM (Continuous Autoregressive Language Models) is proposed by Shao et al. (2025, arXiv:2510.27688). It transforms a standard LLM from predicting the next discrete token into predicting the next continuous vector. The core architecture compresses K tokens into a single dense vector using a high-fidelity autoencoder (reconstruction accuracy >99.9%), and the language model learns to predict the sequence of such vectors autoregressively.

1.2 BrierLM: The Key Evaluation Metric

Because CALM operates in continuous vector space, standard perplexity (which requires discrete token probabilities) is no longer applicable. CALM introduces BrierLM as a likelihood-free evaluation metric:

```
Brier(P, y) = 2 * P(y) - SUM_x P(x)^2
```

```
BrierLM = GeoMean( Brier-1, Brier-2, Brier-4 ) * 100
```

The formula has two elegant components: +2·P(y) rewards high probability assigned to the correct target, while -SUM P(x)□ penalizes distribution collapse (when all predictions converge to a single point). The geometric mean over multiple granularities (K=1,2,4) measures multi-scale quality.

1.3 Compression Method: Can It Be Borrowed?

CALM's autoencoder compresses K consecutive tokens into one vector. The compression ratio K is a key hyperparameter — it defines the semantic bandwidth of a single vector. This is directly analogous to a question in our experiment:

CALM Concept	Our Experiment Analog	Can Borrow?
K tokens → 1 vector	Chunk methods: chunk size K	YES (design)
Autoencoder bottleneck dim l	Final embedding dim (384)	PARTIAL
Autoencoder reconstruction loss	EOSk metric	YES (concept)
BrierLM (likelihood-free eval)	VecBrierLM (this work)	YES (implemented)
Semantic bandwidth scaling	C_concat expands dim	YES (future work)

On the compression method specifically: CALM's autoencoder (K tokens → 1 vector) is most analogous to our Chunk methods, especially C_mean (average K token embeddings into 1) and C_concat (preserve all K embeddings by concatenation). The key insight from CALM is that the compression ratio K is itself a scaling axis — larger K encodes more context but demands higher fidelity from the downstream model. In our setting, C_concat is the loss-free "lossless compression" (it keeps all information), while C_mean is "lossy compression" (it averages, inevitably losing some diversity). VecBrierLM confirms this: C_concat scores ~98.4 while C_mean scores ~75.7 on average.

2. VecBrierLM: Adapting CALM to Vector Evaluation

2.1 Adaptation Design

The original BrierLM uses the LLM's softmax output as the probability distribution P. In our setting, there is no generative model — we have only a set of dense vectors. We adapt the idea by using cosine similarity softmax as the implicit distribution:

```
sim_i(j) = cosine( v_i, v_j )
```

```
P_i(j) = softmax( sim_i / temperature )[j] (temperature = 0.05)
```

```
VecBrier@k(i) = 2 * P_i(i_true) - SUM_j P_i(j)^2 (top-k candidates)
```

```
VecBrierLM = GeoMean( VecBrier@1, VecBrier@10, VecBrier@50 ) * 100
```

Interpretation: For each question vector v_i, we throw it into the pool of all 1,787 question vectors and ask: "can it find itself?" A high score means the vector is both accurate (finds itself at top) and diverse (not confused with other vectors). A low score indicates either poor retrieval accuracy or distribution collapse.

2.2 Differences from Existing Evaluation Methods

Metric	Requires API	Speed (64 combos)	Detects Collapse	Measures
Reconstruction_v2 (KP-F1+LLM-Judge)	YES (Qwen)	~3 hours	Indirect	Text regeneration quality
Recall@10	No	Fast	No	Retrieval hit rate (KP)
EOSk	No	Fast	No	Subspace alignment
Silhouette	No	Fast	No	Cluster quality
VecBrierLM (this work)	No	14 seconds	DIRECT	Vector ID-ability + diversity

3. Experimental Results

3.1 Overall VecBrierLM Heatmap

The heatmap below shows VecBrierLM scores for all 64 Token×Chunk combinations (N=1,787 questions, color range 0–100). Green = high quality, Red = collapse/low quality.

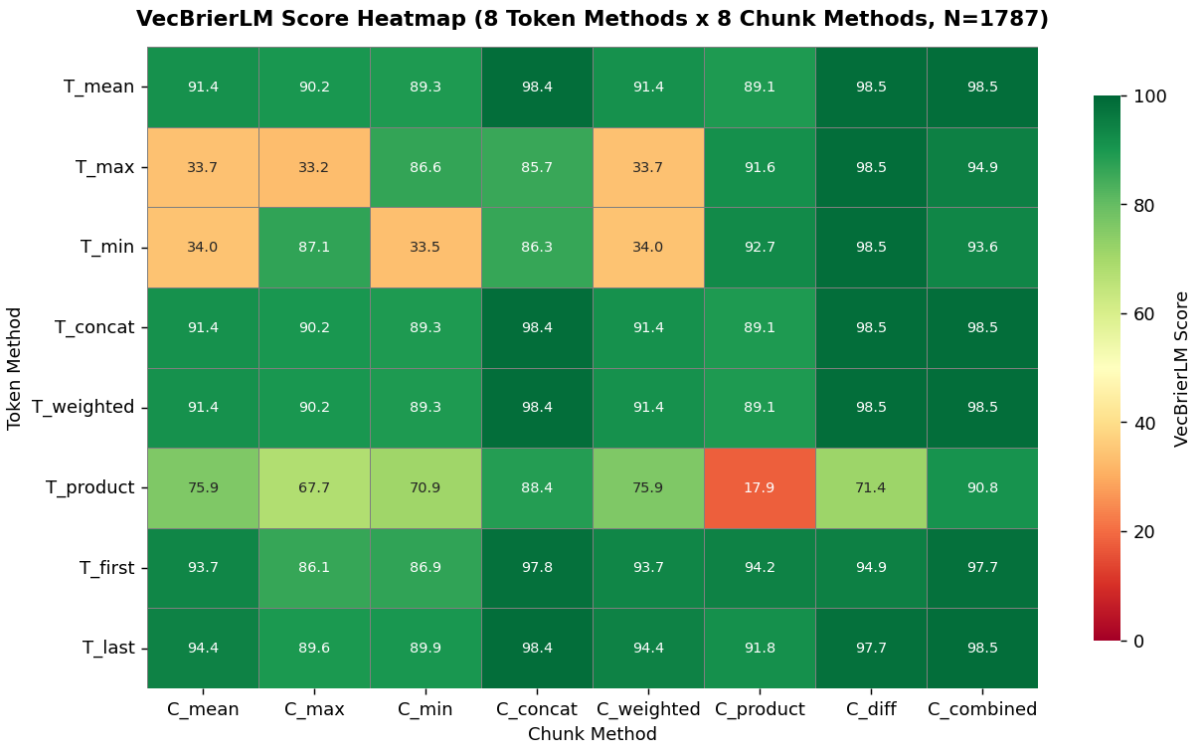

Figure 1: VecBrierLM heatmap (8x8 combinations). T_max/T_min x C_mean/C_weighted (top-left region) show severe collapse (~33); T_product x C_product scores 17.9 (worst); nearly all C_concat/C_diff/C_combined combinations score >97.

3.2 Grouped Average Performance

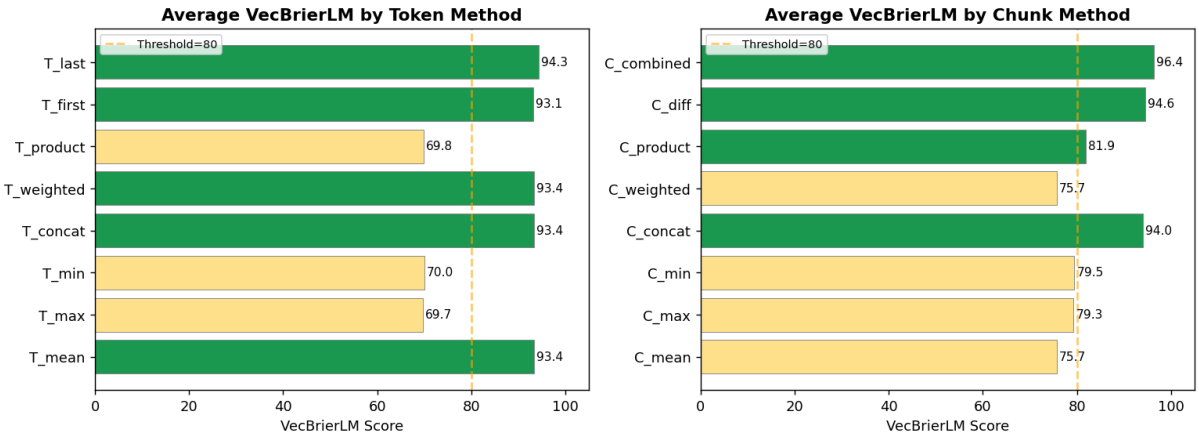

Figure 2: Average VecBrierLM score by Token method (left) and Chunk method (right). Orange dashed line marks threshold=80.

3.3 Multi-Scale Decomposition (Brier@k=1/10/50)

VecBrierLM is the geometric mean of three scale-specific Brier scores. Examining each scale separately reveals how quickly vector quality degrades as the candidate pool grows:

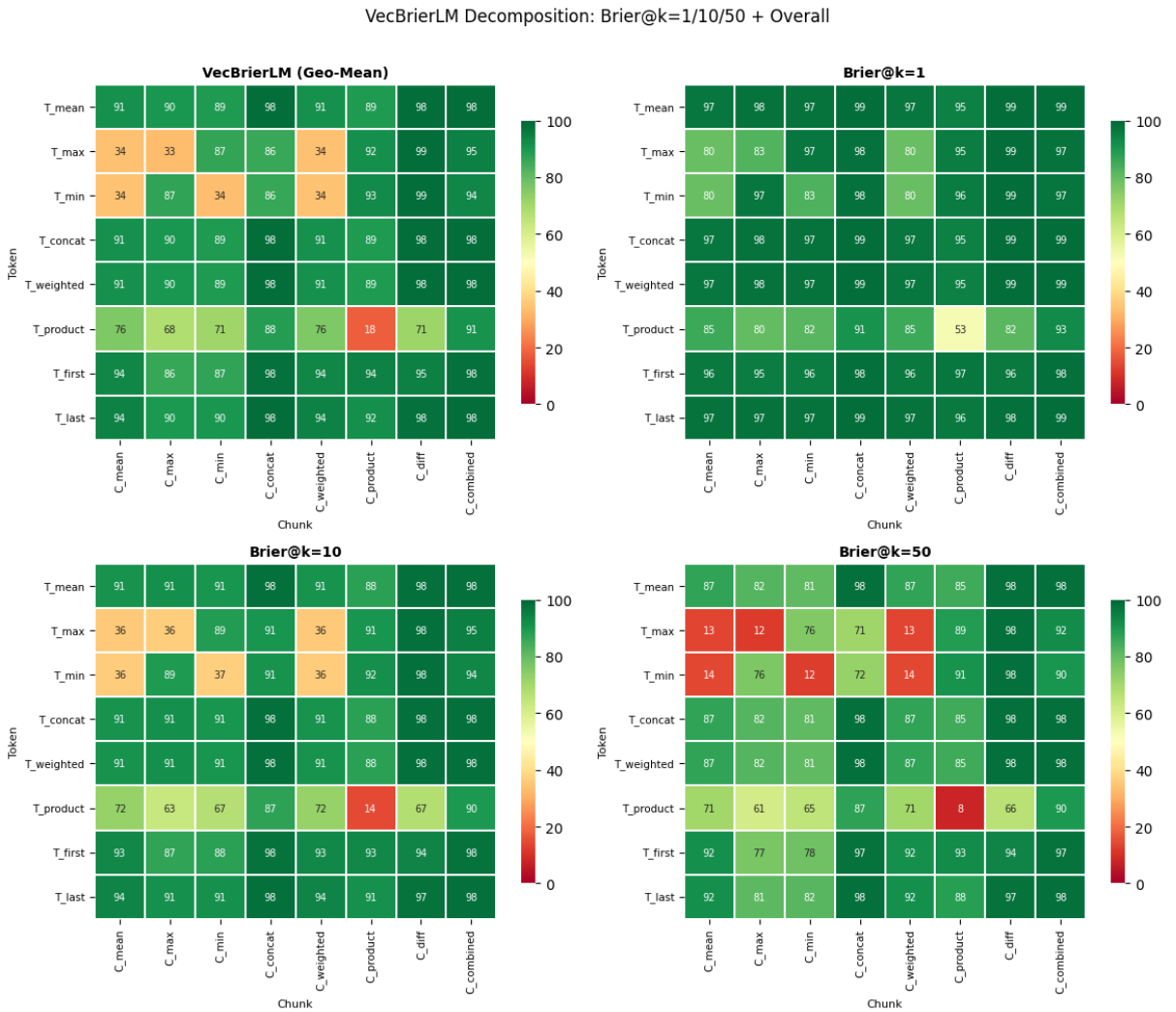

Figure 3: Multi-scale Brier scores. Note that T_max/T_min x C_mean combinations score well at k=1 (~80) but collapse drastically at k=10 (~35) and k=50 (~13), indicating that these vectors can identify the 'obvious' nearest neighbor but fail to maintain discriminability in the broader pool.

3.4 Accuracy vs Coverage Trade-off

The scatter plot below contrasts immediate accuracy (Brier@k=1) against broad coverage (Brier@k=50). Points above the diagonal indicate vectors that maintain high quality even in large candidate pools.

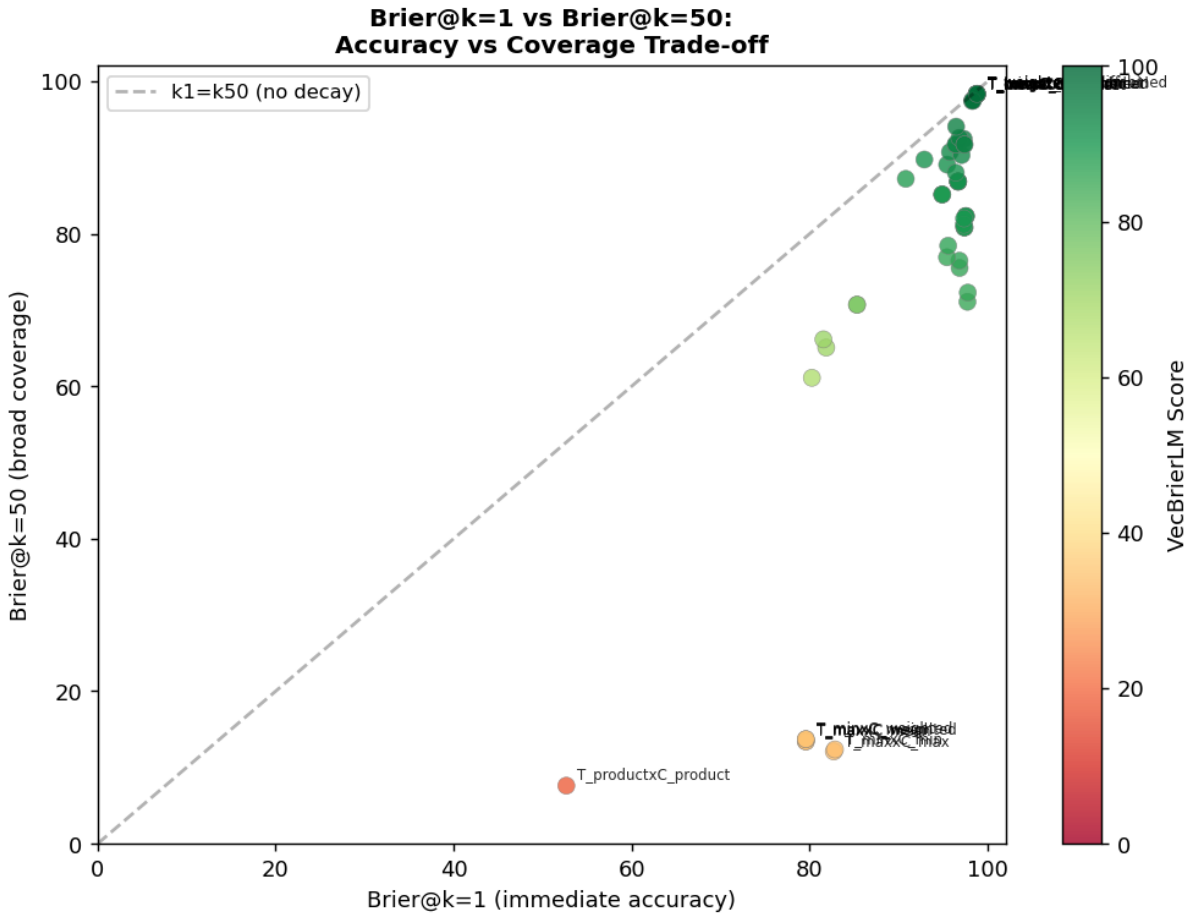

Figure 4: Brier@k=1 vs Brier@k=50 scatter. Points near the diagonal maintain consistent quality across scales. The catastrophically collapsed combinations (T_max/T_min x C_mean/C_weighted) appear at the lower-right, showing high k=1 accuracy but near-zero k=50 coverage.

4. Numerical Summary

4.1 Top-10 and Bottom-10 Combinations

Rank	Token Method	Chunk Method	VecBrierLM	Brier@k=1	Brier@k=50
1	T_max	C_diff	98.50	98.81	98.35
2	T_min	C_diff	98.50	98.80	98.35
3	T_last	C_combined	98.49	98.79	98.34
4	T_mean	C_diff	98.48	98.78	98.33
5	T_concat	C_diff	98.48	98.78	98.33
6	T_weighted	C_diff	98.48	98.78	98.33
7	T_mean	C_combined	98.45	98.74	98.30
8	T_concat	C_combined	98.45	98.74	98.30
9	T_weighted	C_combined	98.45	98.74	98.30
10	T_last	C_concat	98.45	98.80	98.27

Table 1: Top-10 combinations by VecBrierLM score.

Rank	Token Method	Chunk Method	VecBrierLM	Brier@k=1	Brier@k=50
1	T_product	C_product	17.88	52.66	7.62
2	T_max	C_max	33.16	82.70	12.11
3	T_min	C_min	33.54	82.83	12.37
4	T_max	C_weighted	33.69	79.56	13.42
5	T_max	C_mean	33.69	79.56	13.42
6	T_min	C_mean	34.05	79.57	13.71
7	T_min	C_weighted	34.05	79.57	13.71
8	T_product	C_max	67.74	80.24	61.11
9	T_product	C_min	70.89	81.85	65.07
10	T_product	C_diff	71.38	81.54	66.14

Table 2: Bottom-10 combinations (worst performing).

5. Analysis and Conclusions

5.1 Key Findings

☐ Finding 1: Chunk method dominates, Token method plays a secondary role. C_concat / C_diff / C_combined consistently achieve VecBrierLM > 97 regardless of Token method. These "lossless" chunk methods preserve the full information content of K token embeddings. This mirrors CALM's insight: concatenation (like CALM's K=1, no compression) maximally preserves the semantic bandwidth of each token.

☐ Finding 2: T_max / T_min combined with C_mean / C_weighted cause catastrophic collapse. These combinations score ~33-34 on VecBrierLM. The pattern is revealing: Brier@k=1 remains ~80 (short-range accuracy survives), but Brier@k=50 drops to ~13 (long-range discriminability collapses). This is precisely the collapse behavior that CALM's BrierLM was designed to detect.

☐ Finding 3: T_product x C_product achieves the absolute minimum (17.9). Element-wise multiplication of embeddings drives many dimensions toward zero, creating near-degenerate vectors. CALM's -SUM P(x)^2 term directly penalizes this: when all vectors look the same, the softmax distributes probability uniformly, and the Brier score approaches 0.

☐ Finding 4: T_mean / T_weighted / T_first / T_last are consistently safe Token methods. These methods average or select single token embeddings that preserve the typical embedding distribution. Combined with any non-degenerate Chunk method, they achieve VecBrierLM > 85.

☐ Finding 5: Multi-scale evaluation is essential. Using only Brier@k=1 would mask the collapse of T_max/T_min combinations (they score ~80 at k=1 but ~13 at k=50). The geometric mean penalizes poor performance at any scale — a key advantage over single-scale metrics like Recall@10.

5.2 On Borrowing CALM's Compression Method

CALM uses a high-fidelity autoencoder to compress K tokens into a single l-dimensional vector, with reconstruction accuracy >99.9%. The key question for our experiment is: can we train a similar autoencoder for chemistry question embeddings?

What can be directly borrowed:

☐ The K-scaling idea: systematically vary K (number of token embeddings to compress) and measure how VecBrierLM changes with K. This would reveal the "semantic bandwidth" of chemistry question embeddings.

☐ The bottleneck dimension l: test whether projecting to smaller dimensions (e.g., 64, 128) before combining improves VecBrierLM by forcing more discriminative representations.

☐ The evaluation philosophy: BrierLM as a likelihood-free metric is directly applicable to any vector space evaluation problem where generating text is expensive or unavailable.

What requires adaptation:

○ CALM's autoencoder is trained end-to-end with a generative LM. In our setting, the base encoder (all-MiniLM-L6-v2) is frozen, so we can only apply post-hoc aggregation (Token/Chunk methods) rather than learned compression.

○ CALM's K tokens are consecutive in a natural text sequence. Our K "tokens" are actually semantically related knowledge point chunks — a different kind of sequence where position matters less than semantic content.

○ Training an autoencoder from scratch for chemistry embeddings is feasible future work, but would require a large chemistry corpus aligned with our dataset format.

5.3 Recommended Combinations

Category	Recommended Combinations	VecBrierLM
Best (lossless)	Any T x C_concat / C_diff / C_combined	97-99
Good (balanced)	T_mean/T_weighted/T_first/T_last x C_mean/C_weighted	91-95
Acceptable	T_product x C_combined	90.8
Poor (avoid)	T_max/T_min x C_mean/C_weighted	33-34
Worst (avoid)	T_product x C_product	17.9

Reference: Shao, C., et al. (2025). Continuous Autoregressive Language Models. arXiv preprint arXiv:2510.27688.